

CampusConnect: Campus Ride Coordination and Student Safety

NPRT630 / Diploma in ICT / Group Assignment 2026

Final report consolidating problem definition, system analysis, interface and database design, implementation and evaluation. Prepared against the eight-page assignment brief dated 13 February 2026; its due date is TBA. Report preparation date: 5 October 2026.

Submission status: evidence-based draft. The project owner must supply the group number, team leader, all 4–6 full names and student numbers, contribution records, confirmed forked repository URL, human-testing results and final presenter recording. No invented participants or performance measurements appear in this report.

Academic authorship: the group must review and understand the code, diagrams and analysis, cite external material, and follow the institution’s rules on assisted writing. This document is not an attestation of independent authorship.

Member	Full name	Student number / contribution
Team leader	PENDING	PENDING
Member 2	PENDING	PENDING
Member 3	PENDING	PENDING
Member 4	PENDING	PENDING
Member 5, if applicable	PENDING	PENDING
Member 6, if applicable	PENDING	PENDING

Executive overview and report guide

CampusConnect coordinates campus rides for riders, drivers, administrators and security staff. A React Native/Expo client communicates with a Spring Boot REST backend and MySQL. The project brings booking, scheduling, driver decisions, trip status, messages, history, ratings and safety alerts into a single role-based workflow.

The implementation demonstrates a broad student-project feature set. Recent owner feedback led to responsive page frames, one persistent rider navigation bar, clearer back controls and scrollable dialogs. Mocked browser checks validate navigation and selected actions without charging a card or dispatching a real security unit. Nine isolated backend unit tests and a fresh TypeScript check passed on 5 October 2026.

This is not a production assurance report. Biometric matching is not implemented by the inspected face endpoints; payment records do not settle funds; live integration and multi-user usability measurements remain outstanding. Schema drift and privileged account provisioning also need attention.

Coverage	Pages
Phase 1: context, problem, existing approaches, proposed features	3
Phase 2: processes, requirements, planning, UML	4–8
Phase 3: screen catalogue, iterations, ERD and dictionary	9–15
Phase 4: implementation, testing, usability, demonstration	16–19
Limitations, reflection and references	20

Source convention: [B] is the assignment brief; [C] is the current local codebase; [T] is recorded automated evidence; [F] is direct project-owner feedback; [R1–R4] are primary documentation references on page 20. Planning targets and proposed evaluation procedures are not measured outcomes.

Phase 1 — Context, problem and solution

The campus context is Sol Plaatje University, reflected in interface text and configuration. A rider needs a pickup, destination, driver response and a way to report a safety concern. A driver needs a manageable request queue and clear trip controls; administrators need approval and account workflows; security staff need active rides and incident evidence. Campus-wide research has not been supplied, so broader transport behaviour is a design hypothesis rather than a survey finding.

Observed workflow breakdowns [F]: the owner reported that navigation could move with page content, tablet views needed to adjust, and duplicate back buttons were confusing. On 5 October the owner also requested removal of a dollar icon beside a rand-denominated fare. These are direct project feedback, not interviews with multiple students.

Existing approach comparison: phone calls can coordinate a pickup but leave trip status across separate conversations; a group chat can share requests but does not intrinsically provide structured acceptance, scoped ratings and an incident queue; an unrelated transport platform may offer booking but requires investigation of campus membership and security integration. This is a workflow comparison, not a verified market survey. Validate these assumptions with students and campus security.

The proposed improvement is a single journey with explicit statuses and role-appropriate tasks. Success should be measured using booking time, navigation mistakes, task completion and satisfaction before and after the changes; no reduction percentage is claimed.

Concrete functionality	User need
1. Register and sign in; 2. recover password	Access and account recovery
3. Capture a selfie; 4. review driver applications	Onboarding and administration; biometric verification is a gap
5. Request a ride; 6. schedule a ride	Immediate and planned travel
7. Accept/decline requests; 8. update trip status	Driver decisions and rider feedback
9. Exchange ride messages; 10. retain history	Coordination and records
11. Rate a completed ride; 12. manage payment display records	Feedback and payment selection; settlement is out of scope
13. Raise SOS and inspect audio evidence	Safety escalation; actual response is not guaranteed

Phase 2 — Processes and functional requirements

Requirements connect user tasks with inputs, processing and outputs. The IDs below support traceability to code, interface designs and tests. Implementation presence means a route or handler exists; it does not prove that every path works against a live database.

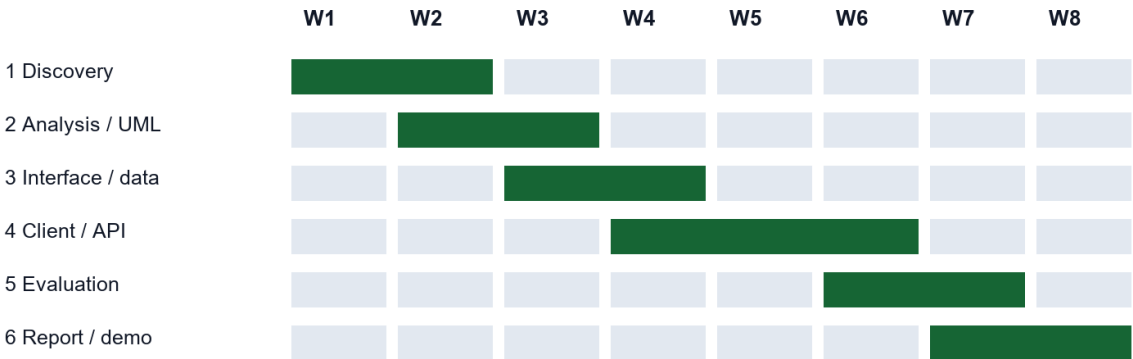
ID / actor	Input → process → output	Code evidence
FR01 / all	Account data → validate and hash password → stored account	AuthController, AuthService
FR02 / all	Email/password → verify and issue JWT → role dashboard	AuthService, SecurityConfig
FR03 / all	Email/code/new password → verify expiry and attempts → password replaced	PasswordResetService
FR04 / applicant	Selfie/direction → accept nonempty upload → success flag	AuthController; no biometric inference
FR05 / admin	Application/decision → update driver approval → queue changes	AdminController
FR06 / rider	Pickup/destination → persist request and fare data → pending ride	RideController, RideService
FR07 / rider	Locations/date/time → store scheduled trip → schedule listing	RideController; dispatch path
FR08 / driver	Request ID/accept or decline → check assignment → ride decision	DriverController
FR09 / driver	Ride ID/status/location → update trip record → rider status	DriverController
FR10 / participants	Ride ID/message → check membership and persist → chat thread	MessageController
FR11 / participants	Completed ride/rating 1–5 → validate ownership → stored rating	RideController
FR12 / rider	Display metadata/method → save owned records → payment preference	UserController, PaymentController
FR13 / rider/security	Ride/GPS/audio → create alert; officer dispatch → incident record	RideController, SecurityController
FR14 / admin	User/status/settings → validate and persist → updated dashboard	AdminController

Booking: rider enters locations → reviews fare → submits → driver reviews → accepts → arrives → starts → completes → participants rate.
 Safety: rider triggers an alert → security inspects context → records dispatch → reviews evidence. A record of dispatch is not proof of a real-world unit response.

Quality requirements and project planning

The targets below are acceptance criteria for future evaluation, not achieved service-level claims. Layout checks already exercise seven viewport sizes; latency, availability and human satisfaction have not been measured. Responsibilities are proposed work packages and must be assigned to named group members.

WBS and illustrative eight-week delivery plan



Proposed relative weeks; actual dates and member ownership are

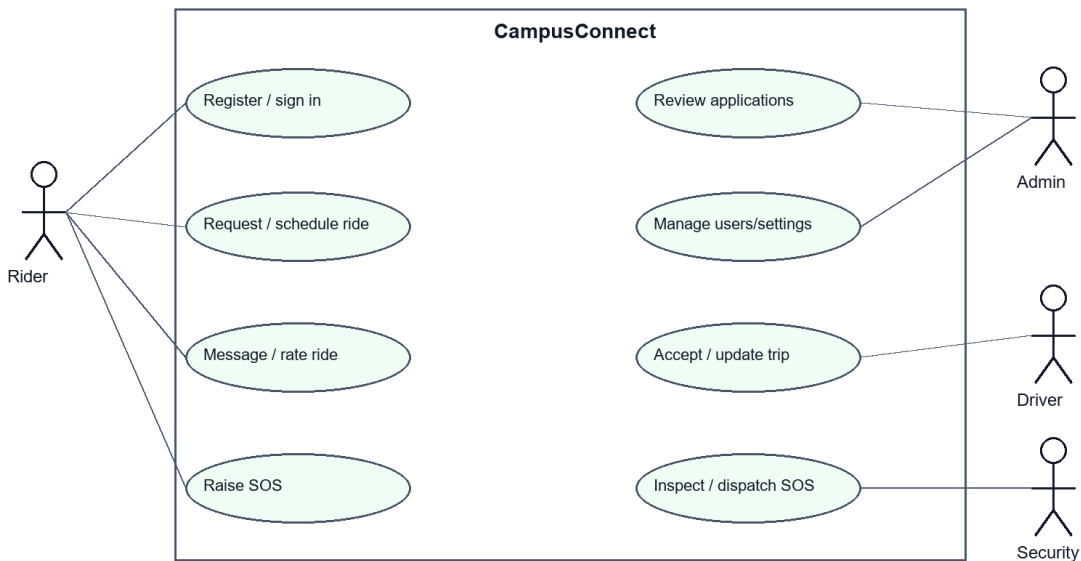
Quality	Measurable target / verification
Reliability	No lost or duplicate booking in 100 controlled attempts; test transaction failures
Availability	Proposed 99% during a 30-day pilot; monitor outages and dependency failures
Performance	Proposed p95 API response <2 s with 20 simultaneous users; controlled load test
Usability / portability	Complete core tasks at 320–1280 px; no hidden footer actions; Android/iOS device checks
Security	401/403 for unauthorised actions; hashed passwords; ownership and role tests
Maintainability	Documented endpoints, repeatable build and migration path; no new critical lint/type errors
Accessibility	Normal text contrast ≥4.5:1; large text ≥3:1; labels and keyboard checks [R1]

WBS: 1 discovery (context and research); 2 analysis (requirements/UML); 3 prototype (screens/data); 4 implementation (client/API/database); 5 evaluation (unit/integration/usability); 6 delivery (report/video/fork). Gantt weeks are an illustrative eight-week plan, not reconstructed actual dates. Confirm baseline, dependencies, owners and actual completion dates before submission.

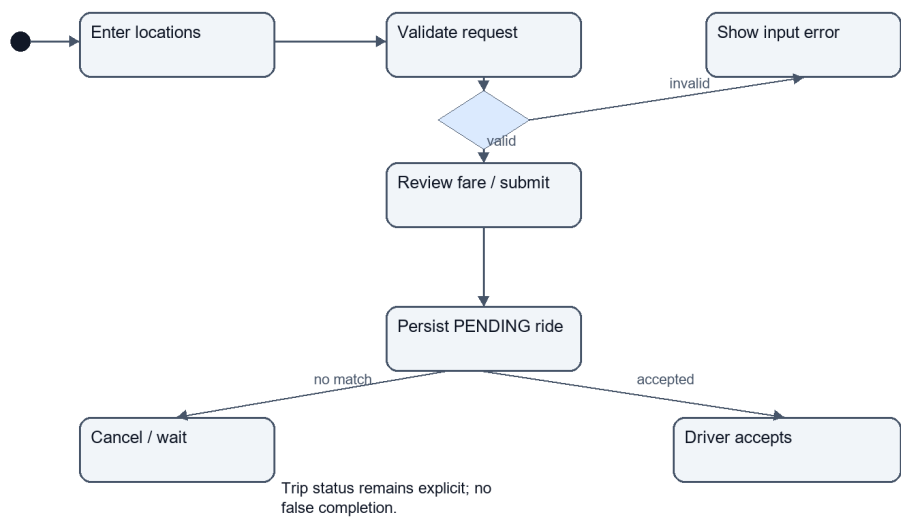
UML — Use cases and booking activity

The use-case diagram presents the actors and their tasks inside the CampusConnect boundary. Authentication supports protected tasks. Administrative approval and security dispatch belong to different roles. The activity diagram describes a rider's immediate-booking journey, including invalid input and unavailable-driver outcomes.

UML use-case diagram — role tasks



UML activity — immediate ride request

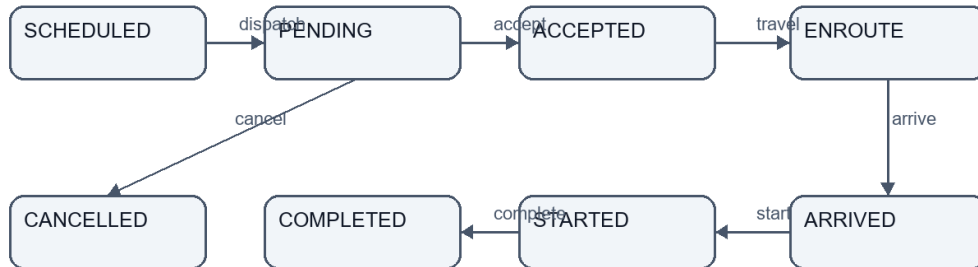


These diagrams document intended interactions supported by the inspected routes [C]. Use-case participation does not establish production authorisation completeness. If no driver accepts, the rider must see the pending state and retain a cancellation path rather than a false completion message.

UML — Trip states and class structure

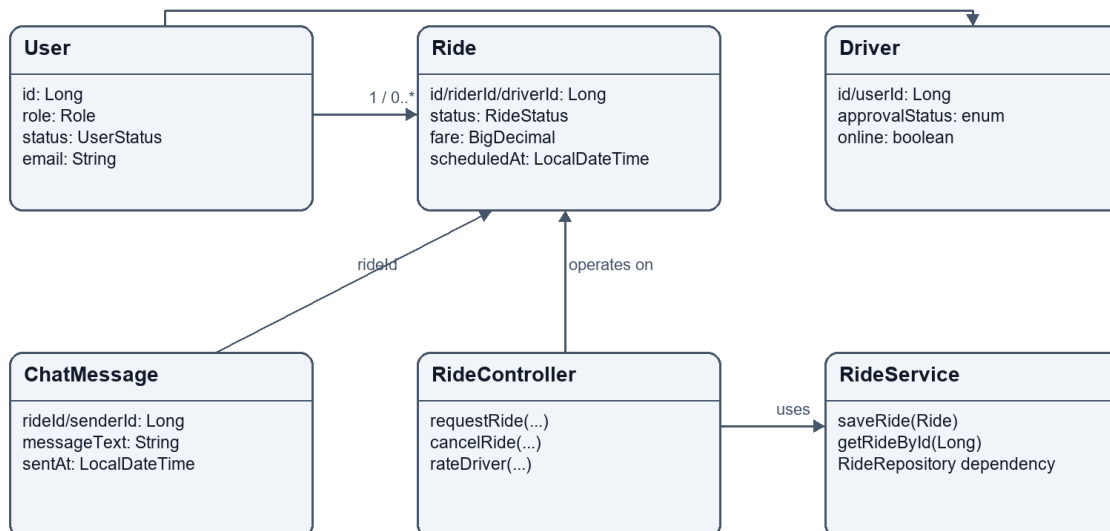
Ride.RideStatus declares SCHEDULED, PENDING, ACCEPTED, ENROUTE, ARRIVED, STARTED, COMPLETED and CANCELLED. The diagram summarises the intended lifecycle, not an assertion that every controller enforces every arrow. A scheduled ride requires dispatch into the request workflow. Cancelled and completed trips remain available for history.

UML state — intended trip lifecycle



Cancellation may occur from permitted active states.
Terminal states retain history; enforcement must be integration-tested.

UML class — selected implementation classes

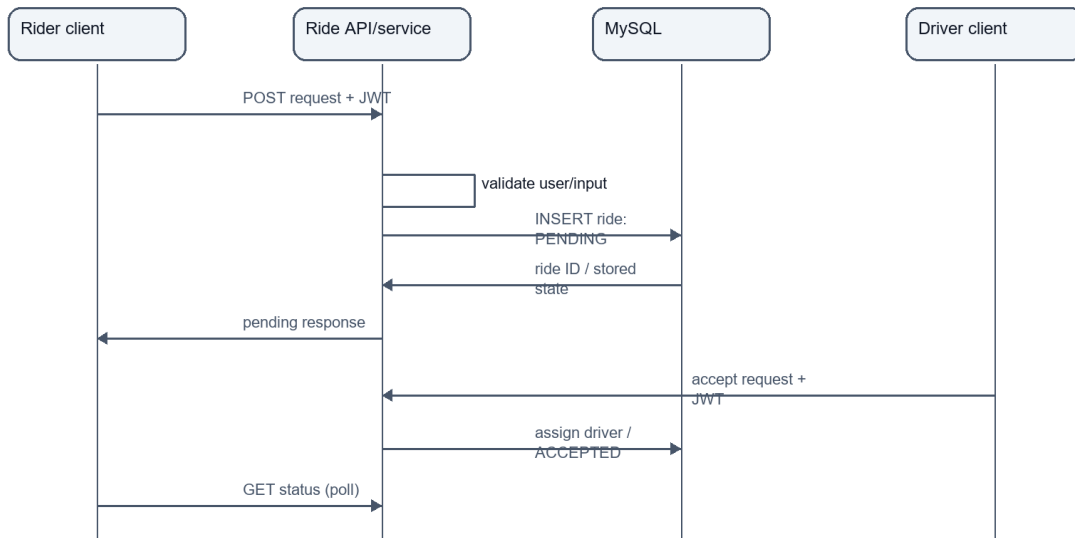


The class diagram represents selected actual Java entities, controller/service boundaries and repositories. Many relationships are stored as scalar IDs rather than JPA object associations. Empty scaffold files, including PaymentService and several proposed entities, are not implemented classes in this model.

UML sequence, architecture and technology

A rider request crosses the client/API/database boundary. A driver's later acceptance changes the stored ride; the client retrieves status through polling. The diagram intentionally depicts polling rather than claiming an active WebSocket delivery path. Failure responses must remain visible to the caller.

UML sequence — request and driver acceptance



Layer / selected technology	Reason and trade-off
React Native 0.86 / React 19.2 / Expo 57	One TypeScript UI with web preview and mobile tooling; native APIs still need device tests [R2]
React Navigation / shared AppLayout	Role routes, safe areas and bounded scrolling; one rider bar persists across four tabs
Spring Boot 3.5 / Java 25	HTTP controllers, dependency injection and security support; versions come from pom.xml [R3]
MySQL / JPA / JdbcTemplate	Relational trip/account data and explicit SQL; InnoDB offers transactions and foreign keys [R4]
JWT / BCrypt / SMTP	Stateless sessions, password hashes and reset email; provisioning and secrets require hardening
Leaflet / OSRM / location / audio	Maps and trip context; availability, permissions and privacy depend on external/device services

Logical architecture: browser/phone → HTTP API → controllers → services/repositories or JdbcTemplate → MySQL. Maps and SMTP are external dependencies. Local API uses port 8080; a physical phone needs a reachable LAN address. HTTPS deployment is a future requirement.

Phase 3 — Authentication and rider entry screens

The catalogue covers every screen registered in mobile/App.tsx. A panel labelled 'SCHEMATIC' is an explanatory mock-up based on route purpose and controls, not a captured runtime screen. 'FIXTURE CAPTURE' identifies an actual local web screen with synthetic data. Captures and mock-ups demonstrate interface design, not real-user evaluation.

Screen catalogue 1 — schematic unless labelled fixture capture

CampusConnect

Brand

loading

sign-in entry

Landing
SCHEMATIC

Welcome Back

Email

Password

Sign In

Recovery

Login
SCHEMATIC

Create Account

Role

contact details

Next

step Back

CreateAccount
SCHEMATIC

Terms and Privacy

Document text

one Back control

Legal
SCHEMATIC

Face Verification

Permission

camera guide

capture

FaceVerification
SCHEMATIC

Responsive

Current Location

Search for a place...

Request Ride

Vehicle Ride

Home
FIXTURE CAPTURE

Ride History

Trips

Rate ride

Previous

Next

Home | History | Schedule | Profile

RiderHistory
SCHEMATIC

Landing shows branding and entry; Login handles credentials and recovery; CreateAccount collects role/contact details and onboarding steps; Legal presents terms/privacy; FaceVerification represents camera permission and selfie capture; Home supports locations and requests; History lists trips with pagination and rating access. Registration step-back is retained; redundant stack headers are suppressed.

Phase 3 — Rider controls and driver entry screens

The four rider tabs share a persistent sibling navigation bar. Their page area shrinks and scrolls above the bar. Secondary screens retain one back control. Shared scrollable dialogs keep actions reachable in short landscape windows.

Screen catalogue 2 — schematic unless labelled fixture capture

Schedule a Ride

Locations

date

time

planned trips

Home | History | Schedule | Profile

RiderSchedule

SCHEMATIC

Profile

Responsive Layout

Student

layout@example.test

2 YEAR

Verified

Edit

\$

cash

Cash

SPU

4 cards

FAVORITES

Promos

8 active promos

Saved cards

Manage payment

Safety

Rate safety

Support

Help center

ACCOUNT

Home

History

Schedule

Profile

RiderProfile

FIXTURE CAPTURE

Payment Methods

Cash

Card

Add card

owned records

RiderPaymentMethods

SCHEMATIC

Rate Your Ride

Five stars

Submit rating

feedback

RatingDriver

SCHEMATIC

Ride Messages

Thread

text input

Send

Close

Chat

SCHEMATIC

Driver Dashboard

Online

requests

View Request

DriverDashboard

SCHEMATIC

Ride Request Details

Rider

route

rand fare

Accept

Decline

ViewRideDetails

SCHEMATIC

ScheduleRide collects route/date/time and lists planned rides; Profile exposes editable personal details and help panels; PaymentMethods saves limited display metadata and default selection; RatingDriver submits 1–5 stars; Chat is a ride-scoped message thread; DriverDashboard shows availability and requests; ViewRideDetails offers request context, rand fare, Decline and Accept Ride. Saved-card storage is not a payment gateway.

CampusConnect | NPRT630 | Evidence-based draft

10 / 20

Phase 3 — Driver and administrative workflows

Driver screens organise the request-to-completion journey and separate earnings/history from the active trip. Administrative screens organise applications, users and current rides. Controls that mutate state display busy feedback; map containers resize when the page dimensions change. Reporting views are dashboard summaries and earnings; no in-app downloadable report-generation screen is evidenced.

Screen catalogue 3 — schematic unless labelled fixture capture

Active Trip

Map

contact

messages

phase button

DriverActiveRide

SCHEMATIC

Earnings

Month comparison

completed-trip totals

DriverEarnings

SCHEMATIC

Trip History

Completed trips

pagination

DriverHistory

SCHEMATIC

Driver Profile

Personal and vehicle details

logout

DriverProfile

SCHEMATIC

Admin Dashboard

Stats

applications

four quick actions

AdminDashboard

SCHEMATIC

User Management

Search

filters

suspend

pagination

UserManagement

SCHEMATIC

Ride Monitoring

Map

current rides

Track Live

Refresh

RideMonitoring

SCHEMATIC

DriverActiveRide exposes trip phases, contact, messages and cancellation; DriverEarnings aggregates completed trips; DriverHistory lists prior rides; DriverProfile presents account/vehicle details; AdminDashboard offers applications and quick actions; UserManagement filters and changes account status; RideMonitoring selects current rides for the map. Aggregated earnings are not confirmation of settlement.

CampusConnect | NPRT630 | Evidence-based draft

11 / 20

Phase 3 — Safety screens and design iterations

Security views prioritise ride context and incident evidence. Incident and resolved-SOS pagination reserve room instead of covering the list. A populated active-SOS list scrolls above its navigation bar. Audio playback and download errors appear as visible feedback.

Screen catalogue 4 — schematic unless labelled fixture capture

Incident Reports

IncidentReports

Audio Recordings

Incident reference

Play

Download

AudioRecordings

SCHEMATIC

The screenshot shows the 'University Settings' app interface. At the top, there's a header with a back arrow, the title 'University Settings', and a subtitle 'Manage different groups and their roles'. A green status bar at the top right shows a signal strength icon and a battery level of 93%. Below the header, there are three main sections: 'INSTITUTIONS', 'COURSES', and 'USERS'. Each section has a list of items with a green arrow icon on the right. The 'INSTITUTIONS' section lists 'University of Cambridge' and 'University of Edinburgh'. The 'COURSES' section lists 'Computer Science' and 'Mathematics'. The 'USERS' section lists 'John Doe' and 'Jane Smith'. At the bottom, there's a green button labeled 'New Settings' and a red button labeled 'Logout User, Supervisor'.

UniversitySettings

Security Dashboard

Safety overview

incidents

navigation

SecurityDashboard

Active Rides Monitor

Map

Center Map

Contact

rides

ActiveRidesMonitor

SOS Alerts

Active alerts

Dispatch

Audio

resolved

SosAlerts

SCHEMATIC

Resolved SOS

Resolved records

search

pagination

ResolvedSos

SCHEMATIC

Connection Test

Diagnostic request

result feedback

TestConnection

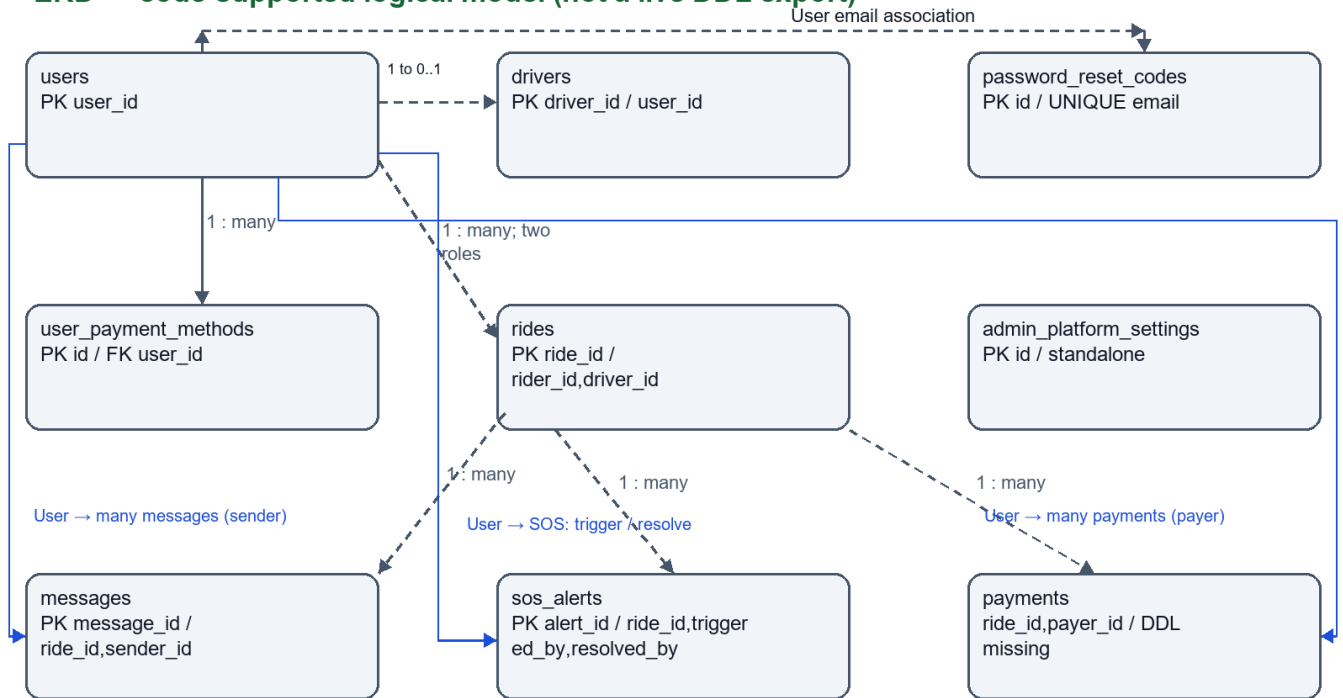
Owner feedback [F] / previous design	Applied revision and evidence
Content displaced bottom navigation	One persistent rider bar; bounded page frame; 28 rider geometry checks [T]
Duplicate or ineffective back controls	Shared header policy; registration step-back retained; Earnings decorative back removed
Dollar icon beside rand fare	DollarSign icon removed from ViewRideDetails on 5 October
Buttons and narrow pages needed care	Scrollable confirmations, mutation guards, toast dismissal and authenticated downloads

Panels include IncidentReports, AudioRecordings, UniversitySettings, SecurityDashboard, ActiveRidesMonitor, SosAlerts, ResolvedSos and the TestConnection diagnostic route. The before/after comparison is reconstructed from recorded code changes and owner requests; no historical screenshot or multi-participant study is invented.

Database design and relationship justification

The ERD distinguishes code-supported logical associations from enforced SQL constraints. The current source is split between JPA entities, JDBC-created tables and historical migrations. It is not a verified export of a live database. Nine tables are represented; empty future entities such as University and SafeZone are excluded.

ERD — code-supported logical model (not a live DDL export)



Grey solid: SQL FK. Grey dashed / blue: code-supported logical associations; verify constraints. Blue links connect User as sender, triggering/resolving officer and payer.

Relationship	Reason / enforcement caveat
User → many rides (rider and driver roles)	A person can participate in multiple trips; driver_id refers to users.user_id, not drivers.driver_id
User → optional driver profile	Intended 1:0..1; Driver.userId alone does not prove UNIQUE enforcement
Ride → many messages / SOS alerts	A trip has a conversation and can have several alerts; sender/trigger IDs reference users
User → many saved-card display records	Owned records, maximum three enforced in application; SQL FK is present
Ride/user → payment records	PaymentController uses ride_id/payer_id; full creation DDL is missing in inspected scripts
Reset code email → user email	Logical association; reset-code table has no declared FK
Platform settings	Standalone intended configuration row; SQL does not enforce singleton count

No standalone many-to-many join table is evidenced. Users can share trips over time through ride participation, but each ride stores one rider and at most one assigned driver. Source details and uncertain field types are recorded in data-dictionary.csv.

Entity dictionary — Accounts, drivers and rides

Field names and sizes below describe the code contract. A grouped row applies the listed type to each field. VARCHAR(255) is the default JPA string length where @Column does not specify a size. The complete one-field-per-row dictionary with source and certainty is supplied as a supporting CSV/Markdown document.

Table / fields	Type / size	Purpose
users: user_id	BIGINT	Primary key
full_name, email, password_hash	VARCHAR(255)	Identity; unique email; password hash
role / status	ENUM	RIDER/DRIVER/ADMIN/SECURITY; ACTIVE/PENDING/SUSPENDED
year_of_study / vehicle_year	INT	Academic and vehicle years
emergency_contact, student_number, phone	VARCHAR(255)	Contact and membership details
licence_plate, vehicle_make	VARCHAR(255)	Registration vehicle details
face_verified / created_at	BOOLEAN / DATETIME	Upload state; account creation
face_embedding / profile_photo	TEXT or LONGTEXT / LONGTEXT	Image data; schema and entity size differ
default_payment_method	VARCHAR(20)	JDBC-added preference
drivers: driver_id / user_id	BIGINT	Driver key and linked user
licence_plate, vehicle_make / vehicle_year	VARCHAR(255) / INT	Vehicle identification
vehicle_photo / rating	LONGTEXT / DECIMAL unspecified	Vehicle image; summary rating
total_trips / online / approval_status	INT / BOOLEAN / ENUM	Activity and PENDING/APPROVED/REJECTED
rides: ride_id, rider_id, driver_id, cancelled_by	BIGINT	Trip key; participant/cancelling IDs
pickup_location, destination	VARCHAR(255) JPA default	Route labels; SQL baseline uses 200
pickup_lat/lng, dest_lat/lng, current_lat/lng	DOUBLE	Coordinates; precision differs in baseline SQL
fare / distance_km / duration_min	DECIMAL unspecified / DOUBLE / DOUBLE	Fare, distance and duration
status	ENUM (8 states)	Lifecycle from Ride.RideStatus
scheduled_at, started_at, completed_at, created_at	DATETIME	Trip timing
rider_rating / driver_rating	INT	Two directions of feedback
rider_rating_comment, driver_rating_comment, cancellation_reason	VARCHAR(500)	Feedback and cancellation explanation

Critical drift: schema.sql calls the ride key id and duration duration_minutes; Ride.java maps ride_id and duration_min. Other scripts use ride_id. Confirm SHOW CREATE TABLE output and consolidate migrations before claiming a clean-install database.

Entity dictionary — Messages, safety and supporting tables

Schema control method: inventory entity mappings and SQL migrations; compare against a disposable database; select one migration history; apply it to an empty database; inspect keys/types; execute registration, trip, chat, SOS and payment tests. Do not run `fix_rides_table.sql` against valuable data: it contains destructive table rebuilding.

MySQL fits relational account/trip records and reporting, while InnoDB supports transactions and referential constraints [R4]. This does not remove the need for correctly declared constraints, tested migrations, backups and least-privilege credentials. No Oracle or NoSQL benchmark was performed.

Table / fields	Type / size	Purpose / source confidence
messages: message_id, ride_id, sender_id	BIGINT	Message and participant keys; entity + migration
message_text / sent_at	VARCHAR(1000) / DATETIME	Body and send time
sos_alerts: alert_id, ride_id, triggered_by	BIGINT	Alert key, trip and triggering user
gps_lat / gps_lng	DOUBLE entity; DECIMAL(10,7) SQL	Coordinates; physical type requires verification
audio_file_path / recording_duration	VARCHAR(500) / INT	Evidence file location and seconds
status / created_at	VARCHAR(30) / DATETIME	Alert state and creation
resolved_at / resolved_by	DATETIME / BIGINT	SQL/controller resolution fields, absent from SosAlert entity
user_payment_methods: id, user_id	BIGINT	Saved display-record ownership
label / last_four / brand / expiry	VARCHAR(80) / CHAR(4) / VARCHAR(32) / CHAR(5)	Card display metadata; no full PAN/CVV
cardholder / is_default / created_at	VARCHAR(120) / BOOLEAN / DATETIME	Display name, default and creation
password_reset_codes: id / email / code_hash	BIGINT / VARCHAR(255) / VARCHAR(255)	Key, unique email and hash
expires_at / created_at / attempts	DATETIME / DATETIME / INT	Expiry, cooldown and invalid attempts
admin_platform_settings: id	BIGINT	Configuration row key; startup DDL
university_name / email_domain	VARCHAR(160) / VARCHAR(120)	Institution display and configured domain
campus_security_phone	VARCHAR(40)	Configured contact
sos_response_time_seconds / first_year_priority_matching	INT / BOOLEAN	Configuration values; not evidence of enforced response/matching
updated_at	DATETIME	Latest configuration save
payments: ride_id, payer_id, amount, method, status, transaction_ref	IDs, decimal amount and strings; sizes UNVERIFIED	Observed JDBC insert contract; creation DDL unavailable

Phase 4 — Implementation and error handling

The client uses registered role routes, Axios API access and shared UI components. AppLayout bounds the navigator, supplies safe-area padding and renders the rider navigation as a sibling. ScrollableCard and ConfirmationDialog keep actions reachable. screenChrome suppresses duplicate native headers where the screen already owns its controls.

The backend exposes authentication, user, driver, ride, message, payment, administrator and security controllers. JPA handles core entities; JdbcTemplate handles saved payment display records, settings and some reporting. This mixed approach makes schema consistency important. Several empty scaffold services/controllers remain and should not be counted as delivered features.

Authentication uses BCrypt and JWT; admin/security paths have role restrictions in SecurityConfig. Password reset stores a hash, expires after ten minutes, uses a five-attempt limit and a sixty-second resend cooldown. Unknown-account requests return without revealing an account. These isolated behaviours are unit-tested; SMTP delivery still needs configuration and live verification.

Failure / action	User-facing behaviour and boundary
Saved-card load/mutation failure	Retry or visible error; busy guard; ownership scoped on server
Dirty settings / leaving	Stay retains edits; Leave replays navigation; save confirms
Missing phone or audio	Visible explanation; no silent disabled control
Audio playback/download	Authenticated request; catch failure; release player on leaving
Trip status / unavailable ride	Visible failure; retry and return path
Network and list loading	Bounded scrolling, loading indicators and selected retry controls

Production blockers found in inspected code: public registration accepts privileged ADMIN/SECURITY role values; startup resets seeded privileged credentials; responses return a User object without evidenced password-field redaction; broad public auth/verification/test paths need review. The report documents these risks without changing application behaviour or printing credentials.

Testing methods, results and colour contrast

Fresh verification on 5 October 2026: npm.cmd run typecheck exited successfully. mvn.cmd -o -Dtest=AuthServiceTest,PasswordResetServiceTest test compiled the project and passed nine tests with zero failures/errors/skips. Repositories and email are mocked in these tests; they do not establish database or SMTP correctness.

Session evidence from 4 October: check-responsive.cjs passed 28 rider tab/size checks, driver/security/admin layout checks, auth forms and dialogs. check-pages.cjs passed seven-size checks for detail pages and mocked card, settings, audio, security, chat and driver-trip actions. Production web export also passed. These are historical recorded checks; they are not newly measured performance or human studies.

Existing historical Surefire reports include one failure in AdminControllerPendingDriversTest (3 tests, 1 failure). Other stored controller reports are not a substitute for a fresh complete run. Some integration tests create accounts, so only the isolated services were rerun for this documentation review.

Colour pair [C]	Calculated contrast	Interpretation [R1]
#111827 on #FFFFFF	17.74:1	Passes normal-text minimum
#6B7280 on #FFFFFF	4.83:1	Passes normal-text minimum
#FFFFFF on #22C55E	2.28:1	Fails normal and large text minima
#FFFFFF on #16A34A	3.30:1	Fails normal; can meet large-text minimum
#FFFFFF on #2563EB	5.17:1	Passes normal-text minimum

Ratios are calculated from theme tokens using relative luminance; they do not constitute a complete accessibility audit. In particular, white text on green buttons needs a darker background or different foreground. The report therefore does not claim WCAG conformance. Test keyboard access, screen-reader labels, zoom, native touch and image text separately.

Human evaluation: protocol and evidence gaps

The brief requires multiple real users, completion times, error rates and satisfaction. No completed participant dataset was supplied. Direct owner requests provide useful formative feedback, but they do not establish a multi-user study or justify numerical usability improvements. This page specifies a reproducible evaluation the group can conduct.

Recruit at least six consenting participants, including rider and driver tasks and an administrator/security reviewer where available. Use anonymous IDs rather than student numbers in results. Explain the test purpose, obtain consent, avoid live emergency dispatch and real card data, and allow participants to withdraw. Report who participated by role without identifying them.

For each task, define the start cue and successful end state before testing. Record seconds, completion without assistance, wrong taps, validation errors, help requests, device/viewport and a 1–5 ease rating. Use equivalent synthetic scenarios for a baseline and revised build, alternate test order when possible, and acknowledge learning effects. Do not invent a baseline if the earlier version is unavailable.

Task	Success state / target to test
Register and sign in	Role dashboard reached; target 90% unaided completion
Request and schedule a trip	Correct route/date saved; target no wrong destination
Find History and return to Home	Correct tab; target no accidental back/scroll obstruction
Driver review/message/start trip	Correct request/status; target no duplicate submission
Edit profile / saved-card record	Validation understood; target save/cancel difference clear
Security inspect a fixture alert	Correct incident/audio; never dispatch a real unit during test

Use supporting usability-results.csv and usability-study.md. Compute completion rate = unaided successes / attempts; error rate = defined error events / attempts; report median task time with sample size and spread. Satisfaction averages are optional and must show raw counts. RESULTS: PENDING. Attach anonymised logs, feedback themes and actual before/after evidence.

Execution, demonstration and delivery

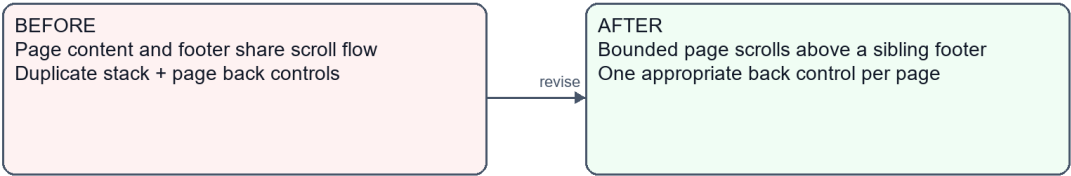
Execution documentation accompanies the report. Install the project’s declared Java 25 and Node dependencies; provide a compatible MySQL database; configure environment secrets locally. The repository uses development ddl-auto update and schema repair, so a clean deployment must verify the schema rather than assume all historical scripts can be run in sequence. Never include real credentials, .env files or local dependency caches in submission.

Run the backend with Maven from backend and the Expo client from mobile. API port is 8080; select a free Expo port such as 8083. ApiClient uses emulator/browser defaults; a physical phone must use a reachable LAN host. Password-reset mail has a separate documented local launcher. A browser build does not prove native camera, microphone or file sharing.

The demonstration script follows a complete user journey: explain the problem → register/login → request/schedule → driver decision → message and trip status → history/rating → admin settings/users → security incident/audio → responsive navigation → limitations and technical decisions. Record the actual integrated system with test accounts for the final presentation. Include presenters’ names and be ready to explain code and answer supervisor questions.

Submission follows the discovered template: Code, Demo, Dox and a names text file, in a module/year/group/leader folder. A code archive is excluded from this working package until it can be exported without secrets/generated caches and tied to the confirmed fork. Code/README gives explicit export instructions. The repository requirement is not fulfilled merely by an existing local git folder.

Design iteration — reconstructed before/after mock-up



Deliverable	Current status
15–20 page report DOCX/PDF	Generated 20-page PDF and editable Word draft
Design and engineering support	UML/ERD images, dictionaries, runbook, API and screen catalogue
Group names and confirmed fork URL	PENDING owner input
Real-user usability results	PENDING data collection
Narrated integrated demo / supervisor Q&A	Script prepared; presenters and final recording pending

Reflection for group review: navigation stability matters because people form spatial expectations; duplicate controls create uncertainty; error messages need an action users can take. Confirm these observations in testing. Add each member’s own account of what surprised them, decisions they made and what they would change.

Limitations, conclusion and references

CampusConnect demonstrates the planned campus workflow through role-specific interfaces and backend routes. Its strongest verified evidence is repeatable layout/action fixtures and isolated account/reset tests. The next priority is an integrated, disposable-database demonstration and an honest human evaluation, followed by native-device checks.

Prioritise: (1) restrict privileged registration and remove credential resets; redact sensitive account fields; (2) consolidate and test schema migrations, including payments; (3) replace selfie acceptance with real verification only if ethically approved and technically validated; (4) enforce lifecycle/assignment concurrency and evaluate external dependencies; (5) improve text contrast, keyboard access and native permissions; (6) measure real-user outcomes and record a narrated demonstration. No payment settlement, biometric assurance, campus-wide adoption, uptime or guaranteed emergency response is claimed.

This report is ready for group review but must not be described as submission-complete while names, fork identity, contribution evidence, participant results and the final demo are missing. The verified template structure is used; update folder/file names once the group number and leader are confirmed.

Reference	Source / use
[B] Assignment brief	2026-Group-Assignment_NPRT630-Project.pdf, 13 Feb 2026, pp.1–8; requirements and rubric
[C] Local project	backend/src, mobile/src, database/mysql, pom.xml, package.json; inspected 5 Oct 2026
[T] Test evidence	Surefire service reports (5 Oct); session browser/build checks (4 Oct); test scripts and docs/responsive-layout.md
[F] Owner feedback	Conversation requests on responsiveness/back controls; fare icon request on 5 Oct 2026
[R1] W3C	https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html
[R2] Expo	https://docs.expo.dev/
[R3] Spring Boot	https://docs.spring.io/spring-boot/index.html
[R4] MySQL InnoDB	https://dev.mysql.com/doc/refman/8.4/en/innodb-introduction.html

External references accessed 5 October 2026. Version numbers are taken from the project manifests rather than inferred from current vendor documentation. Supporting files contain source paths, reproducible commands and explicit evidence gaps. Replace draft metadata and add members' own final reflection before submitting.